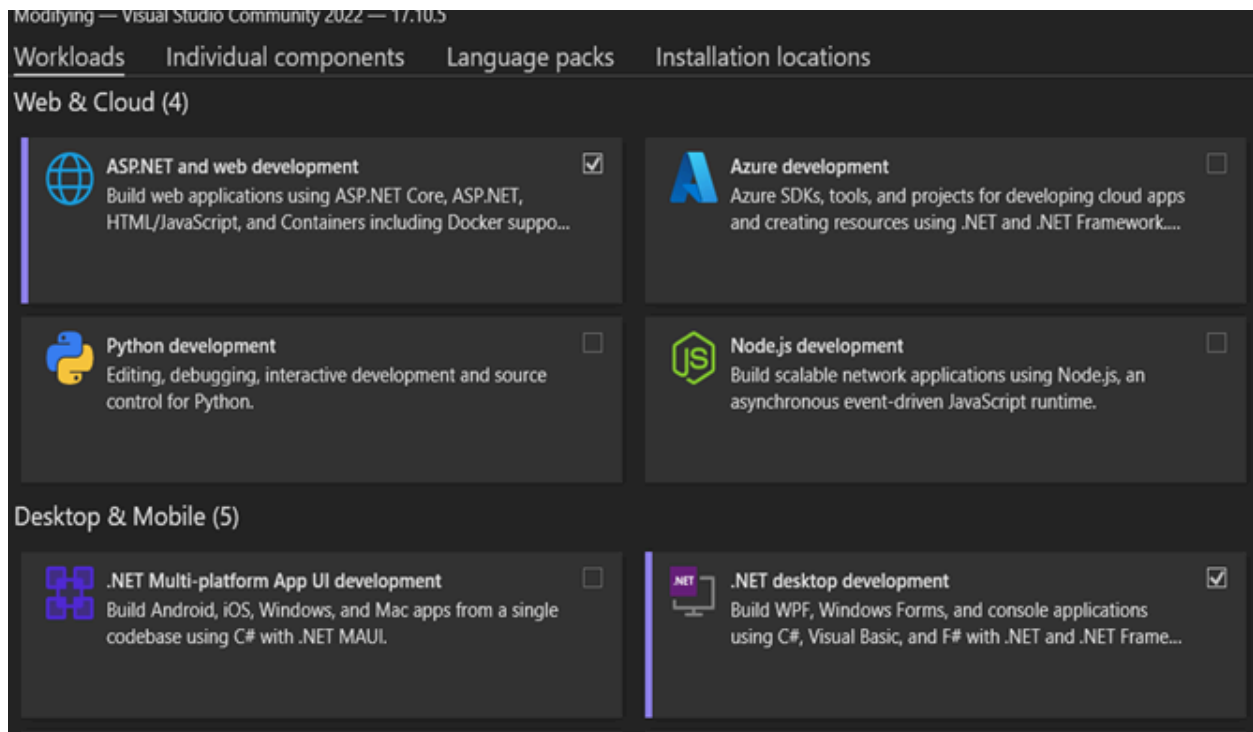


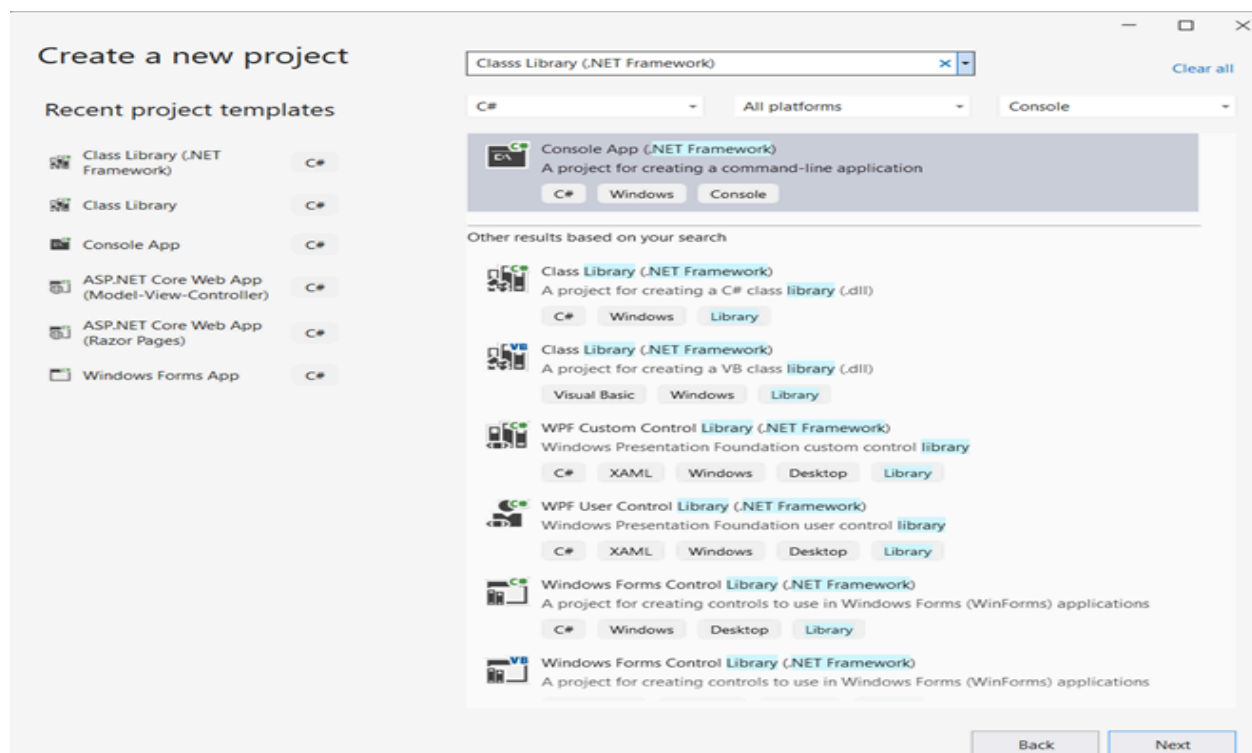
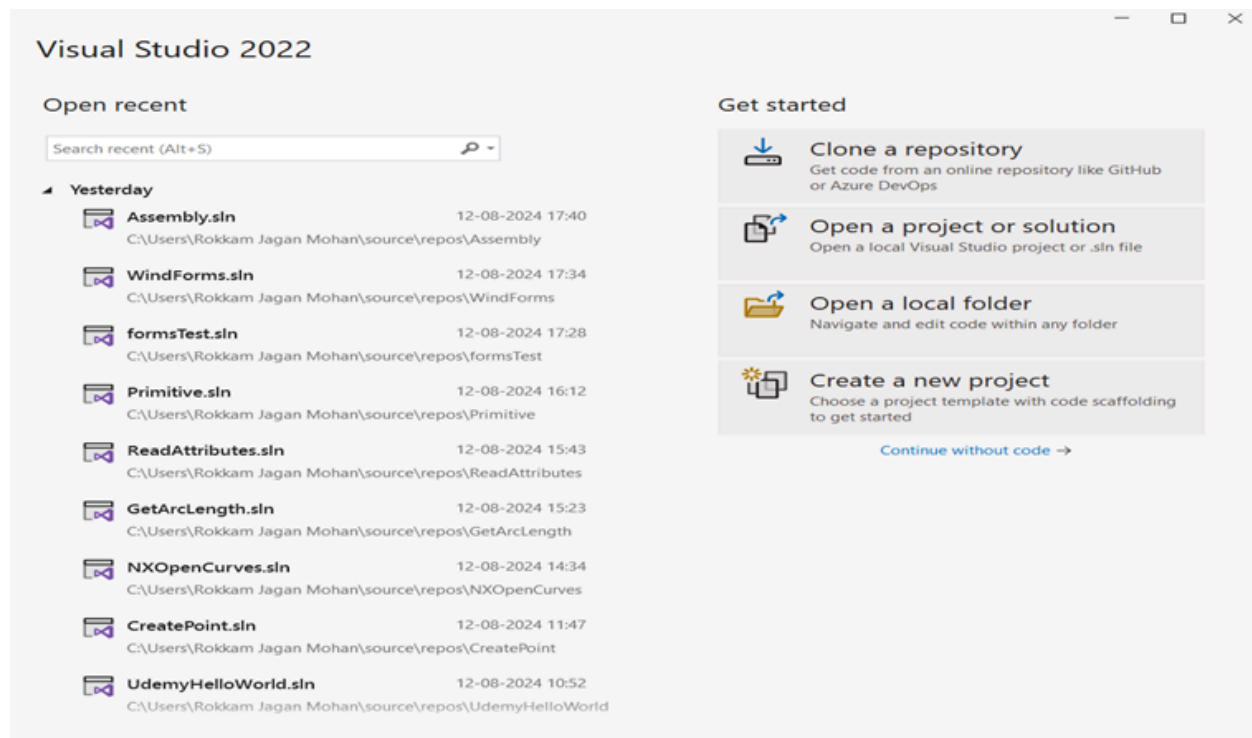
NX Open Customization

PROJECT CREATION :

- Install Visual Studio Community. Run it.
- Add assemblies as selected below which will be used as Virtual Run Time machines for our code.

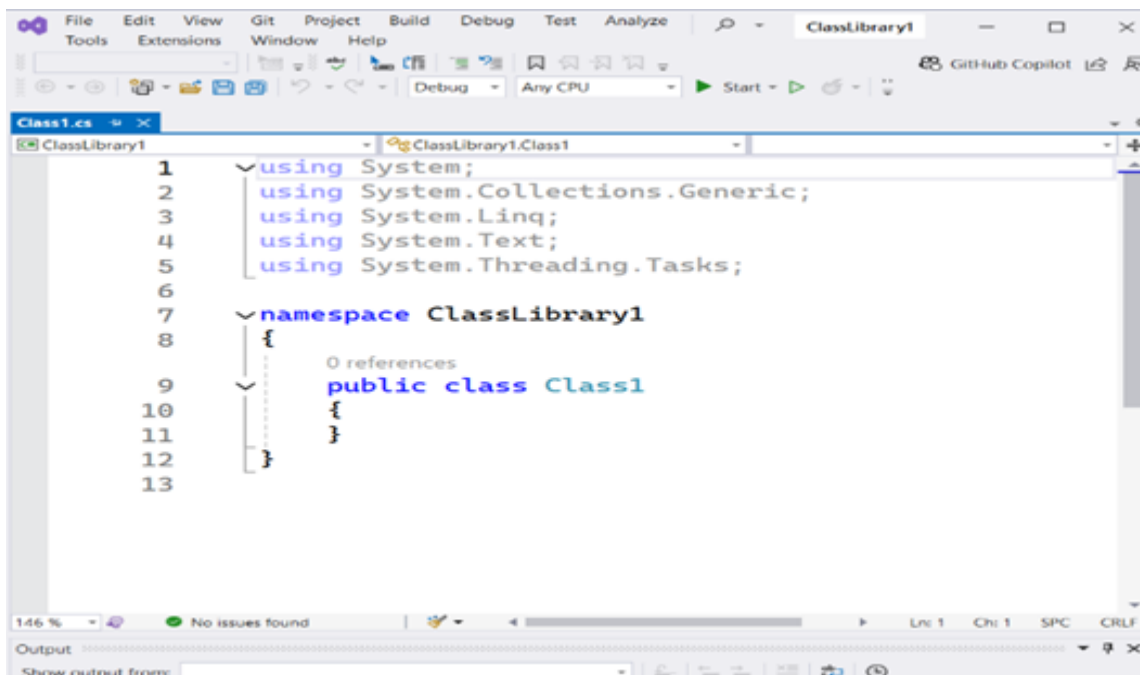


- We use (.dll) extension files to communicate with NXOpen through our code which means whatever code we write in Visual Studio (C#) that should be converted and saved as .dll files.
- In order to start coding, select create new project in Visual studio.

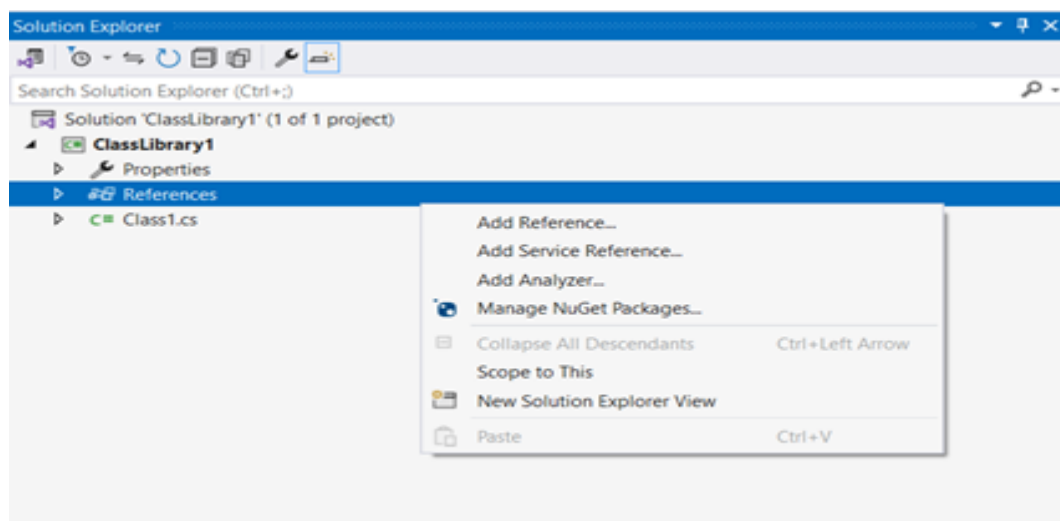


Name the project according to your choice and you can change the location if you want then click create.

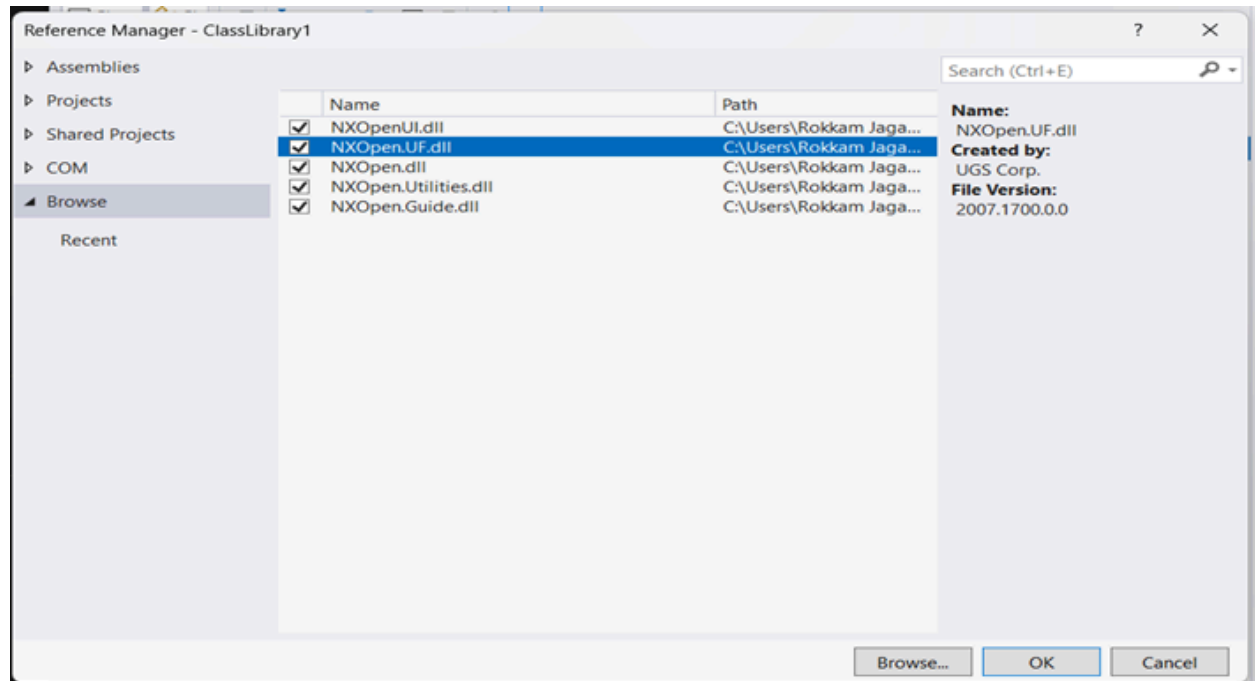
Then VS will create a new project workspace for us and it will provide a basic C# template like below.



- Open Solution explorer(ctrl + Alt + L), give a right click on references and select add references. These references are NXOpen libraries from which we can use the predefined methods and classes of NXOpen.

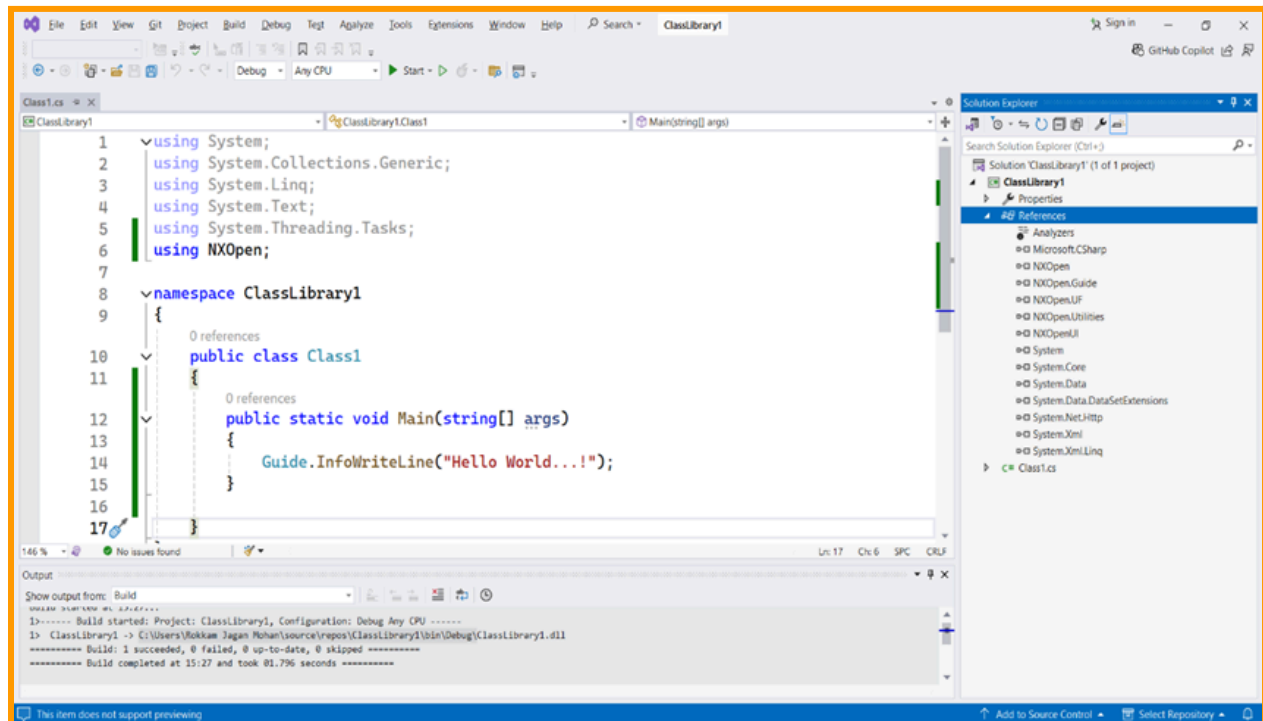


- We should add the dll files which are given by NX software. The location of the files will be Program Files/Siemens/NX2007 /NX BIN/managed. There will be plenty of files shown. You should select only the below mentioned dlls. Select them and Click add.



- After successful adding of references we can start writing c# code to customize NXOpen.

Here is a sample Hello World program which will be printed in NX.



Step 1 : After adding references type “using NXOpen” in line 6.

Step 2 : line 12 is the Main method() which will be the entry point of program execution.

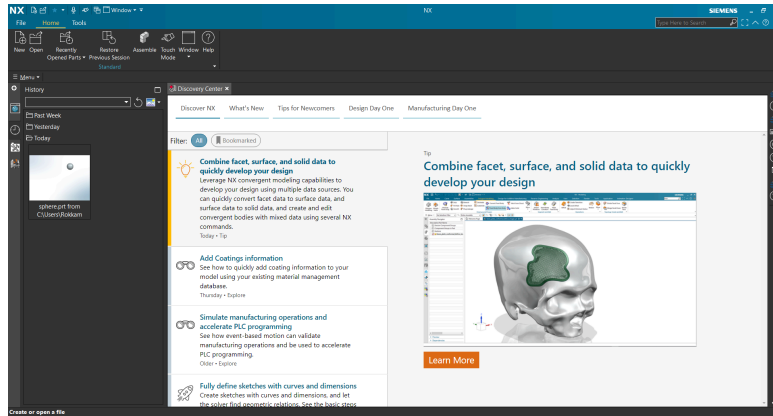
Step 3 : Line 14 allows us to print something in the listing window in NX which will be seen in the output below. Guide.InfoWriteLine acts as Console.WriteLine.

After writing the code as above we want to build the code in order to convert it into a dll file.

So give a right click on the project name ClassLibrary1 in the solution explorer and select build. Or you can simply use Ctrl+B to build your project.

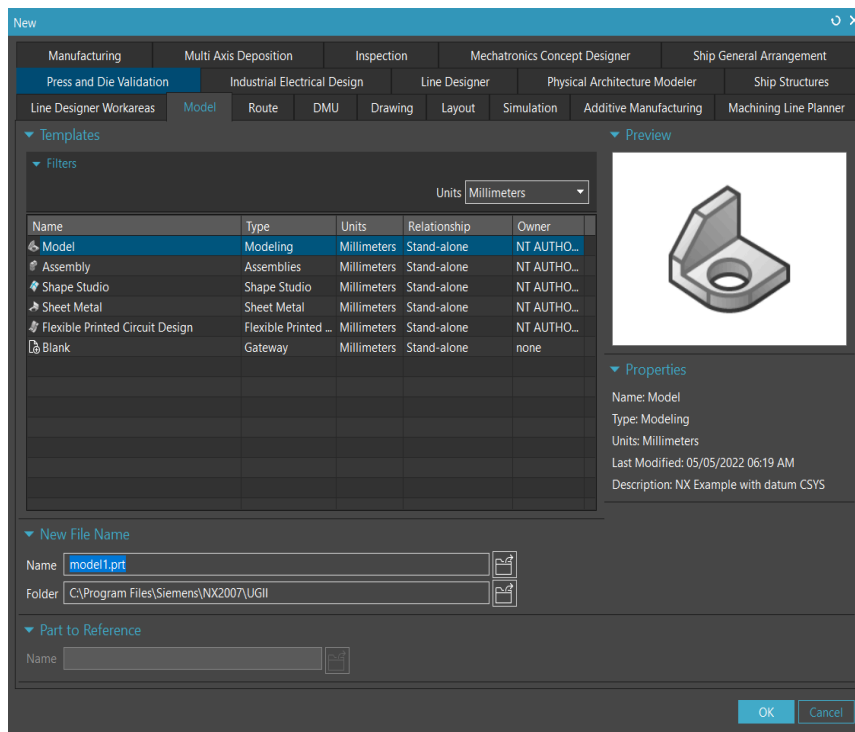
It will result in a successful build when there are no errors in the code. As given in the above picture.

Copy the path until Debug and Open NX App.



Select Model and name it according to your need and change location if you want.

Click ok.

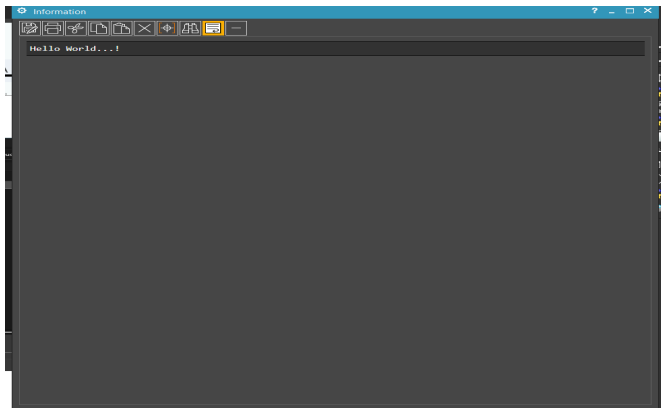


You will get a empty work space so you are all set to run your hello world

Click on File, select Execute, NXOpen or you can simply press Ctrl+U.

Now paste the earlier copied path and search it. Now select the dll with your project name. For now we named it as default ClassLibrary1 so lets select that and click ok.

So we will get output in the listing window like below.



Now lets go and look at some NXOpen stuff below.

Positions Vectors and Points in NXOpen

1. After Creating .Net Class Library file
2. Add in the references the following dlls:
 - NXOpen.dll
 - NXOpen.Guide.dll
 - NXOpen.UF.dll
 - NXOpen.Utilities.dll
 - NXOpenUI.dll
3. Code goes Like this:

A screenshot of a Visual Studio code editor window. The file name is "PointTutorial.cs". The code is written in C# and includes several using statements at the top: System, System.Collections.Generic, System.Linq, System.Text, System.Threading.Tasks, and NXOpen. The code defines a namespace "CreatePoint" containing a public class "PointTutorial". Inside the class, there is a public static void "Main" method that takes a string array "args" as a parameter. The "Main" method contains several lines of code: it gets the active session of NX, creates a part, gets the points collection, creates a new Point3d object, creates a point from the collection, and sets the visibility of the point to visible. The code is formatted with line numbers on the left side of the editor.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using NXOpen;
7
8  namespace CreatePoint
9  {
10     0 references
11     public class PointTutorial
12     {
13         0 references
14         public static void Main(string[] args)
15         {
16             //Get active session of NX
17             Session session = Session.GetSession();
18
19             //Create Part
20             Part workPart = session.Parts.Work;
21
22             PointCollection points = workPart.Points;
23
24             Point3d coords = new Point3d(3,10,0);
25
26             //Point point = points.CreatePoint(coords);
27
28             Point point = workPart.Points.CreatePoint(coords);
29
30             point.SetVisibility(SmartObject.VisibilityOption.Visible);
31         }
32     }
33 }
```

Understanding the code:

"using NXOpen;":

It is used to include all the methods and instances of NXOpen Library.

Session session = Session.Session();

It is used to get the active session of the nx model file.

Part workPart = session.Parts.Work;

It is used to get the active Work Part(Working Space) of the nx model file.

Point3d coords = new Point3d(3, 5, 0);

It defines the coordinates of a point in 3D (x=3 units,y=5units,z=0 units)

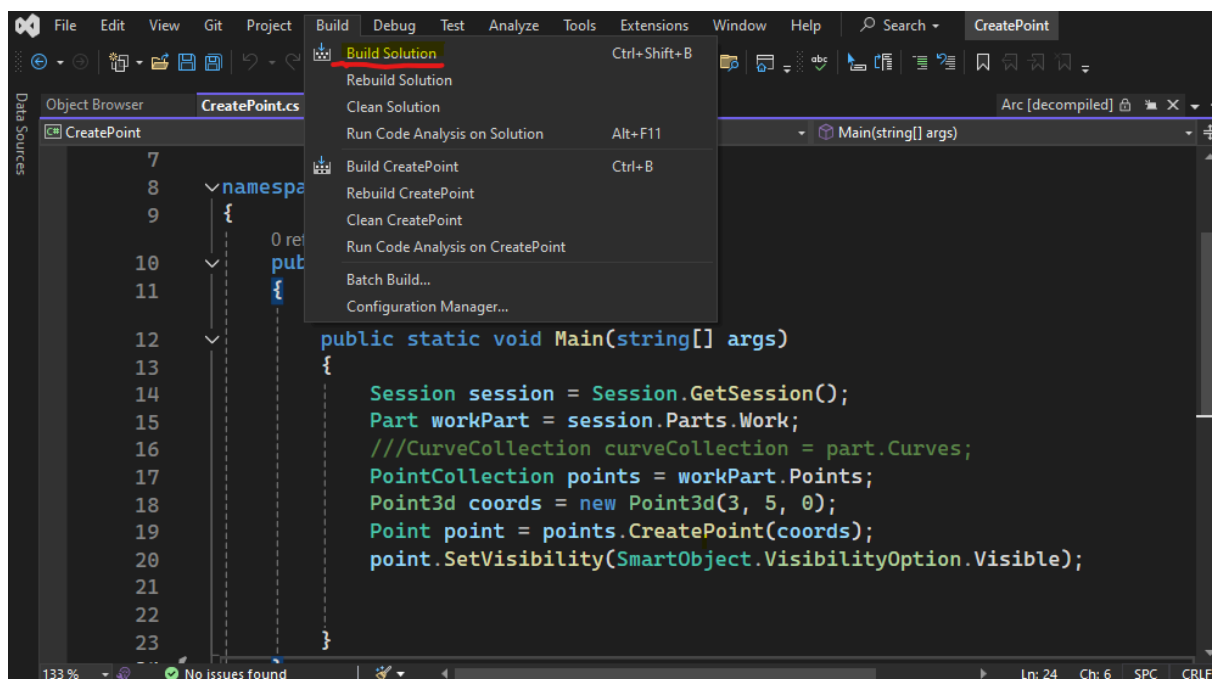
Point point = points.CreatePoint(coords);

It creates the point at the mentioned "coords" (added to the collection)

point.SetVisibility(SmartObject.VisibilityOption.Visible);

This Line is necessary to include as NXOpen.Point is a "SmartObject" which is invisible by default. So we need to manually set the visibility ON

4.Build the file to get the .dll file:



5. Open NX App => File => Execute => Browse for the corresponding .dll files(Path will be like : C:\Users\ADMIN\source\repos\CreatePoint\bin\Debug\CreatePoint.dll):

Output will be :



Don't worry it got created but not visible just Zoom it and take a look.

Working with curves and sketches

1. Select .Net class library file and write the following code after adding the references :

```
LinkageTutorial.cs  + X
C# LinkageBar LinkageBar.LinkageTutorial
5 using System.Threading.Tasks;
6 using NXOpen;
7
8 namespace LinkageBar
9 {
10     0 references
11     public class LinkageTutorial
12     {
13         0 references
14         public static void Main(string[] args)
15         {
16             //Get active session of NX
17             Session session = Session.GetSession();
18
19             Part part = session.Parts.Work;
20
21             CurveCollection curveCollection = part.Curves;
22
23             //Parameters
24             double length = 8, width = 4, half = width / 2, holeDiam = half, pi = Math.PI, twopi = 2 * pi;
25
26             Point3d p1, p2, p3, q1, q2, q3;
27
28             p1 = new Point3d(0, -half, 0);
29             p2 = new Point3d(0, 0, 0);
```

```

    p2 = new Point3d(0, 0, 0);

    p3 = new Point3d(0, half, 0);

    q1 = new Point3d(0, -half, length);

    q2 = new Point3d(0, 0, length);

    q3 = new Point3d(0, half, length);

    //Create lines
    curveCollection.CreateLine(p1, q1);

    curveCollection.CreateLine(p3, q3);

    Vector3d axisX = new Vector3d(0, 1, 0);

    Vector3d axisY = new Vector3d(0, 0, 1);

    //Create arcs
    curveCollection.CreateArc(q2, axisX, axisY, half, 0, pi);

    curveCollection.CreateArc(p2, axisX, axisY, half, pi, twopi);

    //Create Holes
    curveCollection.CreateArc(q2, axisX, axisY, holeDiam / 2, 0, twopi);

    curveCollection.CreateArc(p2, axisX, axisY, holeDiam / 2, 0, twopi);
}
}

```

Understanding the code:

```
CurveCollection curveCollection = part.Curves;
```

It creates an instance of a Curve Object in the Workpart Area.

```
p1 = new Point3d(0, -half, 0);
```

It defines a 3D coordinate in the workpart.

(X=0, y=-half = -2, z= 0);

```
curveCollection.CreateLine(p1, q1);
```

It creates a line having starting point p1(0,-2,0) and ending point q1(0,-1,8)

```
Vector3d axisX = new Vector3d(1, 0, 0);
```

It defines a unit vector along the x- axis direction.

```
curveCollection.CreateArc(q2, axisX, axisY, half, 0, pi);
```



`Arc CurveCollection.CreateArc(Point3d center, Vector3d xDirection, Vector3d yDirection, double radius, double startAngle, double endAngle)`
(+ 2 overloads)

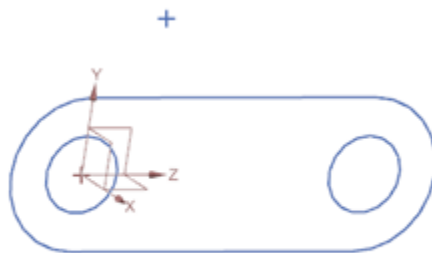
Creates an `NXOpen.Arc` .

The arc will be created through the origin and whose normal is Z axis.
The start and end angles are measured relative to the X and Y axes.

Created in NX4.0.0

License requirements: `solid_modeling` ("SOLIDS MODELING") OR
`drafting` ("DRAFTING") OR `geometric_tol` ("GDT")

Output:

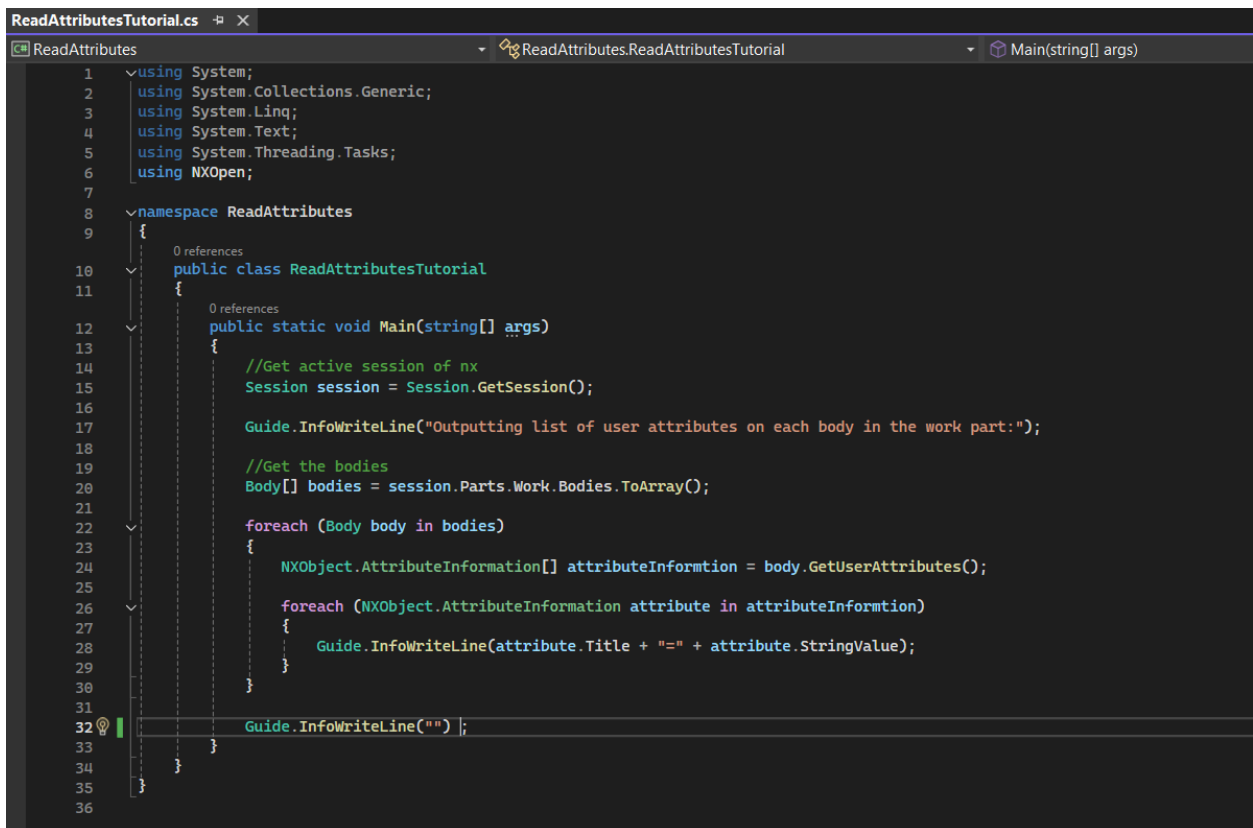


Read Attributes From Files

- It is about how to grab attributes values from a particular work.
- Attributes are nothing but the file properties that all the materials that have been used for the particular file whether it is made or by part , that is created by whom, approved by whom, created on which date like these information are added in the attributes.

Basic Program for Read Attributes

1. First Follow the Steps in the Project Creation Section.
2. Generate Session.



```
ReadAttributesTutorial.cs  ReadAttributesTutorial  Main(string[] args)
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using NXOpen;
7
8  namespace ReadAttributes
9  {
10     0 references
11     public class ReadAttributesTutorial
12     {
13         0 references
14         public static void Main(string[] args)
15         {
16             //Get active session of nx
17             Session session = Session.GetSession();
18
19             Guide.InfoWriteLine("Outputting list of user attributes on each body in the work part:");
20
21             //Get the bodies
22             Body[] bodies = session.Parts.Work.Bodies.ToArray();
23
24             foreach (Body body in bodies)
25             {
26                 NXObject.AttributeInformation[] attributeInformtion = body.GetUserAttributes();
27
28                 foreach (NXObject.AttributeInformation attribute in attributeInformtion)
29                 {
30                     Guide.InfoWriteLine(attribute.Title + "=" + attribute.StringValue);
31                 }
32             }
33
34             Guide.InfoWriteLine("");
35         }
36     }
```

Working:

- It is Console which opens in the NX Open which is called the Listing Window. It shows the text written here.

```
Guide.InfoWriteLine("Outputting list of user attributes on each body in the work part:");
```

- This line retrieves all the bodies (solid objects, surfaces, etc.) in the currently active (work) part. `session.Parts.Work` accesses the currently active part, and `.Bodies.ToArray()` converts the collection of bodies into an array.

```
//Get the bodies
Body[] bodies = session.Parts.Work.Bodies.ToArray();
```

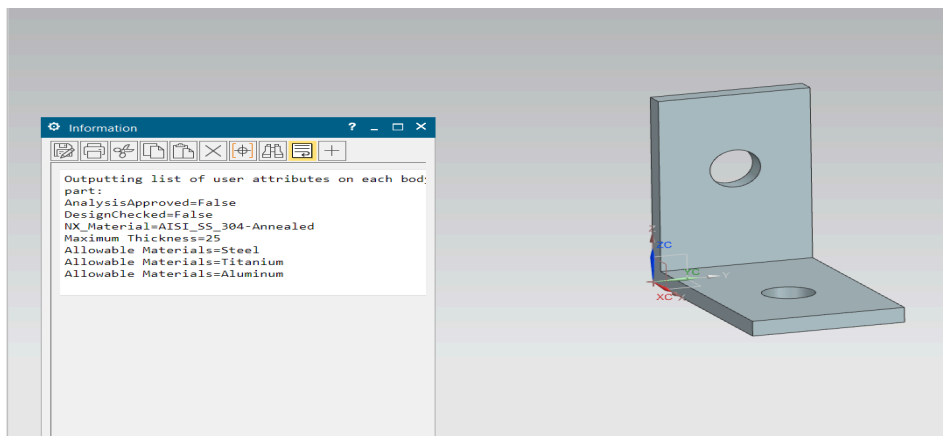
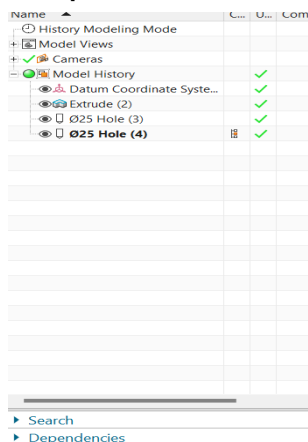
- The below lines explain that the code iterates over each body in the bodies array.
- For each body, the code checks if there are any user-defined attributes (like custom labels or properties) attached to that body and it is done by a running method which is `body.GetUserAttributes()`.
- The code then goes through each of these attributes one by one.
- For each attribute, the code prints out the name (or title) of the attribute and its value (the information stored in that attribute) in the format "Title=Value."

```
foreach (Body body in bodies)
{
    NXObject.AttributeInformation[] attributeInformation = body.GetUserAttributes();

    foreach (NXObject.AttributeInformation attribute in attributeInformation)
    {
        Guide.InfoWriteLine(attribute.Title + "=" + attribute.StringValue);
    }
}

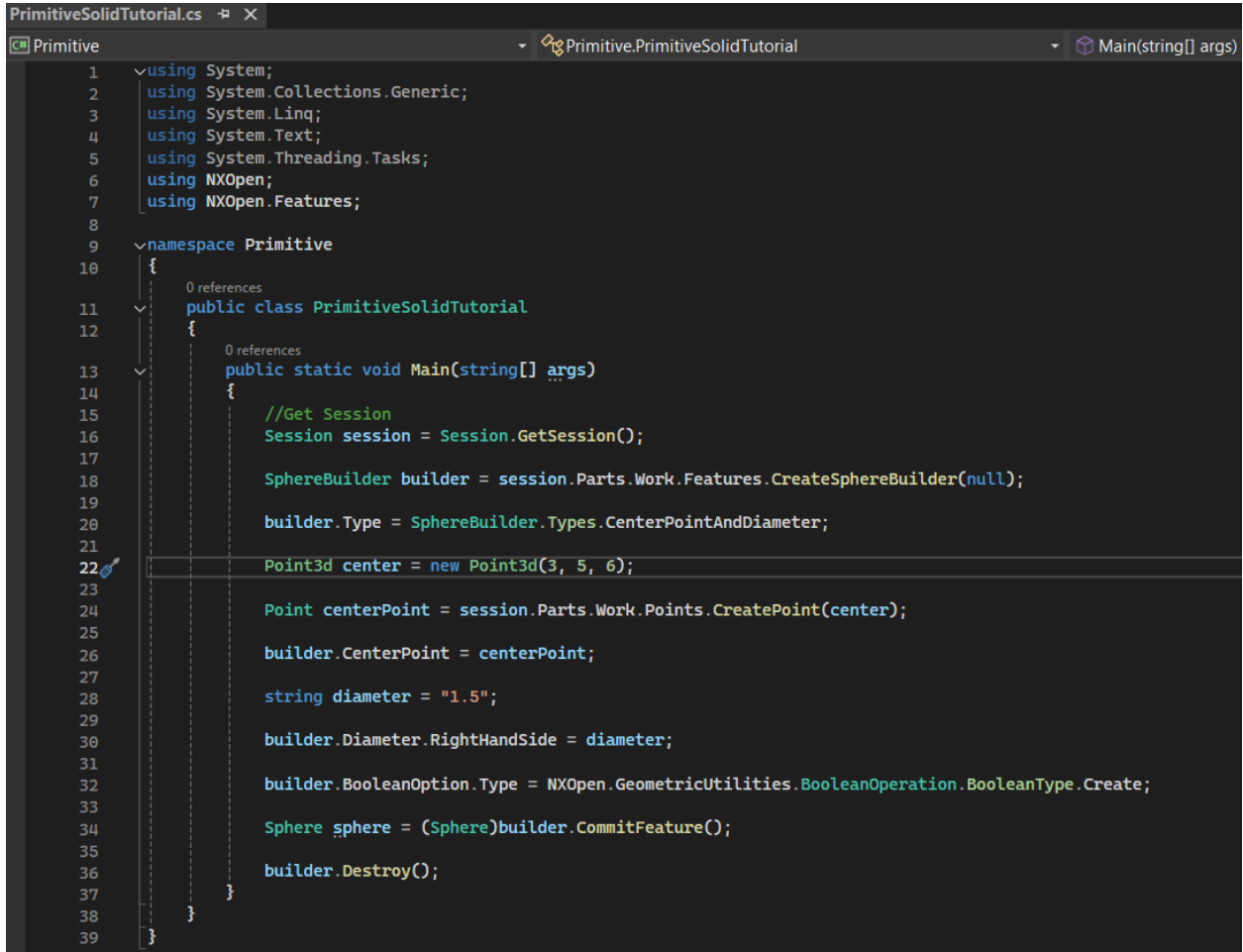
Guide.InfoWriteLine("") ;
```

Output:



Creating Solid Bodies

- It is about creating primitive solid bodies.
- The NXOpen.Features class provides a variety of functions for creating simple solid primitive features (blocks, cylinders, cones, spheres, etc.)
- As an example, let's consider the following code that builds a sphere feature:



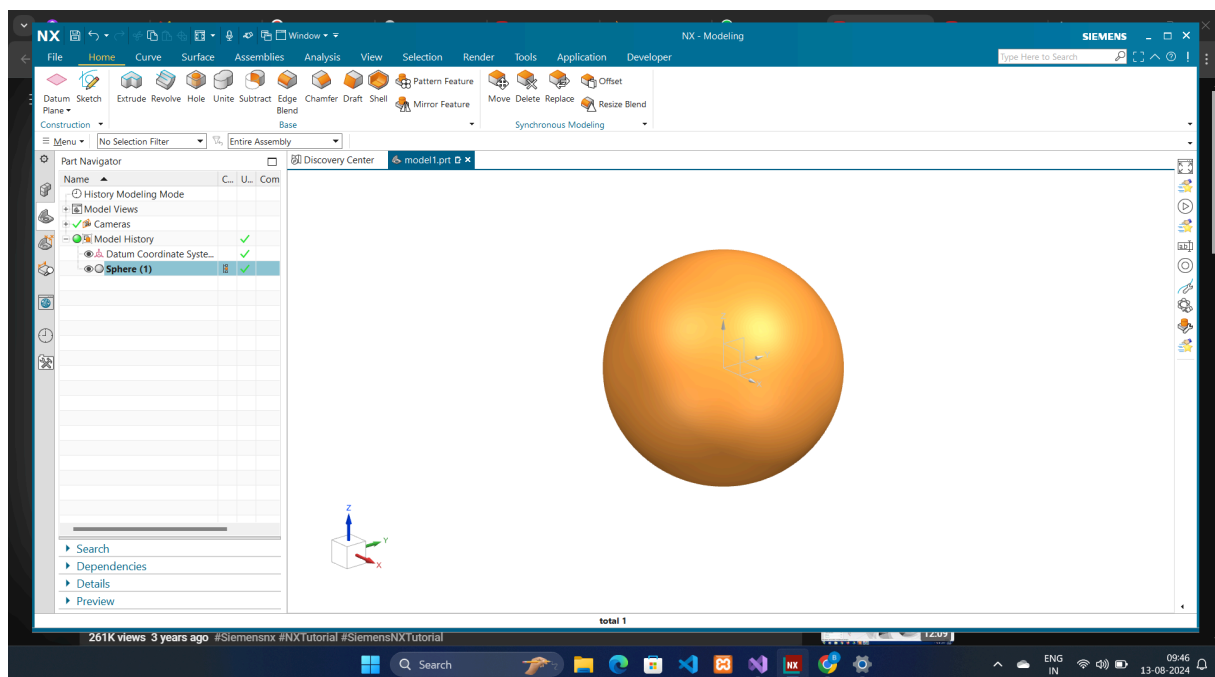
```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using NXOpen;
7  using NXOpen.Features;
8
9  namespace Primitive
10 {
11     public class PrimitiveSolidTutorial
12     {
13         public static void Main(string[] args)
14         {
15             //Get Session
16             Session session = Session.GetSession();
17
18             SphereBuilder builder = session.Parts.Work.Features.CreateSphereBuilder(null);
19
20             builder.Type = SphereBuilder.Types.CenterPointAndDiameter;
21
22             Point3d center = new Point3d(3, 5, 6);
23
24             Point centerPoint = session.Parts.Work.Points.CreatePoint(center);
25
26             builder.CenterPoint = centerPoint;
27
28             string diameter = "1.5";
29
30             builder.Diameter.RightHandSide = diameter;
31
32             builder.BooleanOption.Type = NXOpen.GeometricUtilities.BooleanOperation.BooleanType.Create;
33
34             Sphere sphere = (Sphere)builder.CommitFeature();
35
36             builder.Destroy();
37         }
38     }
39 }
```

Working:

1. First Follow the Steps in the Project Creation Section.
2. Generate Session.
3. In line-18 SphereBuilder is a tool for specifying the properties of the Sphere like center point, radius. 'CreateSphereBuilder' is a method that creates a new SphereBuilder object.
4. In line-18 Features is a property of the Work part, which provides access to various feature creation tools (like creating spheres, extrusions, holes, etc.).
5. The line-20 specifies the sphere definition type (center and diameter).

6. The line-22 defines the Sphere Center.
7. The line-32 defines the boolean option (create, unite, etc.). We will get the idea of these terms when we use NX Open Manually.
8. The line-34 Commit to create the feature.
9. So, the basic format is:
 - Create a “builder” object
 - Modify its properties as desired
 - “Commit” the builder to create a new feature

OUTPUT:



NXOpen :

NXOpen is Siemens NX's API framework that allows developers to automate and customize NX using various programming languages, including C#, VB.NET, and Python. Here's a basic introduction to the key classes and components in NXOpen:

1. Session Class

- The Session class represents the active NX session. It provides access to many high-level operations like getting the current work part, creating new parts, and handling user interaction.
- **Key Methods/Properties:**
 - Session.GetSession(): Retrieves the current NX session.
 - Session.Parts.Work: Returns the current work part.
 - Session.MessageBox(): Displays messages to the user.

2. Part Class

- The **Part** class represents a part file within NX. You can interact with this class to modify geometry, perform analyses, and manage assemblies.
- Key Methods/Properties:
- Part.Features: Provides access to all the features in the part.
- Part.Bodies: Returns the solid bodies in the part.

3. UI Class

- The UI class is used to manage the user interface, handle selection, and display messages or dialog boxes.
- Key Methods/Properties:
- UI.GetUI(): Retrieves the instance of the UI class.
- UI.SelectionManager: Manages user selections.

4. NXObject Class

- The base class for all NX objects. Any entity in NX, such as points, curves, and bodies, inherits from this class.
- Key Methods/Properties:
- NXObject.GetTag(): Retrieves the tag (unique identifier) of an NX object.
- NXObject.SetName(): Sets the name of an NX object.

5. Features Class

- The Features class represents features in a part, such as extrusions, holes, and patterns.
- Key Methods/Properties:
- Features.FeatureCollection(): Provides access to the features in a part.
- Features.ExtrudeBuilder(): Used to create extrusion features.

6. Builder Classes

- Builder classes are used to construct features or modify objects. These classes simplify the creation of complex features.
- Example: ExtrudeBuilder, HoleBuilder.

7. SelectionManager Class

- Manages the selection of objects by the user. This is critical for interacting with geometry or entities in the UI.
- Key Methods/Properties:
- SelectionManager.SelectObject(): Prompts the user to select an object.

8. UFSession Class

- UFSession (Unigraphics Functions) provides access to low-level UFUNC commands. It's often used for legacy commands or more complex operations that aren't available through NXOpen.
- Key Methods/Properties:
- UFSession.GetUFSession(): Gets the UFSession instance.

9. BlockStyler Class

- BlockStyler is used to create custom dialog boxes and UI forms within NX.
- Key Methods/Properties:
- BlockDialog: Represents a custom dialog box created via BlockStyler.

10. NXException Class

- Handles exceptions that may occur during the execution of NXOpen code. This is important for error handling and debugging.
- Key Methods/Properties:
 - NXException.ErrorCode: Retrieves the error code associated with an exception.

Each class has its own set of methods and properties, and they interoperate to give you control over the NX environment. Depending on what tasks you want to automate or customize, you'll need to explore specific classes more deeply.